
ARTIFICIAL INTELLIGENCE

Lab Report — 2

First Order Predicate Logic & Prolog Programming



TRIBHUVAN UNIVERSITY

Institute of Engineering, Pulchowk Campus

Student Name: Aditya Kumar Shah
Roll Number: 080BCT011
Batch: BCT III/II
Department: Electronics & Computer Engineering
Subject: Artificial Intelligence (CT653)
Date: 2083-03-16

Contents

1	Objectives	3
2	Theoretical Background	3
2.1	First Order Predicate Logic	3
2.2	FOPL to Prolog Mapping	3
2.3	Inference Strategies	3
3	Assignment 1 — Course Preference via Resolution	4
3.1	Problem Summary	4
3.2	FOPL Representation	4
3.3	Resolution Proof	4
3.4	Prolog Code	4
3.5	Output	4
3.6	Discussion	5
4	Assignment 2 — Horses, Mammals, and Inverse Relations	6
4.1	Problem Summary	6
4.2	FOPL Representation	6
4.3	Prolog Code	6
4.4	Inference Trace	6
4.5	Output	7
4.6	Discussion	7
5	Assignment 3 — Student Evaluation: Forward & Backward Chaining	8
5.1	Problem Summary	8
5.2	FOPL Representation	8
5.3	Prolog Code (Backward Chaining)	8
5.4	Backward Chaining Trace	8
5.5	Forward Chaining Trace	9
5.6	Forward Chaining in Prolog	9
5.7	Output	9
5.8	Discussion	10
6	Summary	11
7	Conclusion	11
8	References	11

1 Objectives

1. To translate natural-language knowledge statements into First Order Predicate Logic (FOPL).
2. To implement FOPL rules as Prolog programs and verify queries through backward chaining.
3. To demonstrate both *forward chaining* and *backward chaining* on an inference problem.
4. To apply resolution and rule-chaining techniques to answer queries in a knowledge base.

2 Theoretical Background

2.1 First Order Predicate Logic

First Order Predicate Logic (FOPL) extends propositional logic by introducing predicates, quantifiers, and variables. It provides a formal framework for representing knowledge about objects and their relationships. The key components are: predicates (e.g. $\text{Horse}(x)$), constants (e.g. *charlie*), variables (e.g. X), universal quantifier ($\forall$: “for all”), and existential quantifier ($\exists$: “there exists”).

2.2 FOPL to Prolog Mapping

Prolog is a computational realisation of Horn-clause logic. Universal quantifiers become Prolog variables, implications become rules ($:-$), conjunctions become comma-separated sub-goals, and ground facts represent existential or specific assertions.

2.3 Inference Strategies

Backward chaining (Prolog’s default) starts from the goal and works backwards to facts via unification and backtracking. **Forward chaining** starts from known facts and applies rules iteratively to derive new conclusions. Both are sound and complete for Horn-clause knowledge bases.

3 Assignment 1 — Course Preference via Resolution

3.1 Problem Summary

A knowledge base describes Steve's course preferences. Steve only likes easy courses; science courses are hard; basket weaving courses are easy; and BK301 is a basket weaving course. The task is to use resolution to determine which course Steve would like.

3.2 FOPL Representation

1. $\forall X (\text{course}(X) \wedge \text{easy}(X) \Rightarrow \text{likes}(\text{steve}, X))$
 2. $\forall X (\text{science_course}(X) \Rightarrow \text{hard}(X))$
 3. $\forall X (\text{basket_weaving_course}(X) \Rightarrow \text{easy}(X))$
 4. $\text{basket_weaving_course}(\text{bk301})$
 5. $\text{course}(\text{bk301})$
-

3.3 Resolution Proof

1. $\text{basket_weaving_course}(\text{bk301})$ — given fact.
2. By rule (3): $\text{easy}(\text{bk301})$.
3. $\text{course}(\text{bk301})$ — given fact.
4. By rule (1): $\text{course}(\text{bk301}) \wedge \text{easy}(\text{bk301}) \Rightarrow \text{likes}(\text{steve}, \text{bk301})$.
5. $\therefore \text{likes}(\text{steve}, \text{bk301})$. ■

3.4 Prolog Code

```

1 course(bk301).
2 basket_weaving_course(bk301).
3
4 likes(steve, X) :-
5     course(X),
6     easy(X).
7
8 hard(X) :-
9     science_course(X).
10
11 easy(X) :-
12     basket_weaving_course(X).
```

Listing 1: Course preference resolution

3.5 Output

```

?- likes(steve, X).
X = bk301.
```

```
?- likes(steve, bk301).  
true.
```

Listing 2: Assignment 1 output

3.6 Discussion

Resolution chains the basket-weaving fact through the “easy” rule and combines it with the “course” fact to satisfy Steve’s preference condition. Prolog returns **X = bk301** via automatic backward chaining and unification.

4 Assignment 2 — Horses, Mammals, and Inverse Relations

4.1 Problem Summary

Five statements about horses, mammals, offspring, and parent relations are given. The goal is to determine whether Charlie is a horse, using the facts that Bluebeard is Charlie's parent and Bluebeard is a horse.

4.2 FOPL Representation

1. $\forall X (\text{Horse}(X) \Rightarrow \text{Mammal}(X))$
 2. $\forall X, Y (\text{Offspring}(X, Y) \wedge \text{Horse}(Y) \Rightarrow \text{Horse}(X))$
 3. $\text{Parent}(\text{bluebeard}, \text{charlie})$
 4. $\forall X, Y (\text{Offspring}(X, Y) \Leftrightarrow \text{Parent}(Y, X))$
 5. $\forall X (\text{Mammal}(X) \Rightarrow \exists Y \text{Parent}(Y, X))$
-

4.3 Prolog Code

```

1 parent(bluebeard, charlie).
2 horse(bluebeard).
3
4 mammal(X) :- horse(X).
5
6 horse(X) :-
7     offspring(X, Y),
8     horse(Y).
9
10 offspring(X, Y) :- parent(Y, X).
```

Listing 3: Horses and mammals with inverse relations

4.4 Inference Trace

```

?- horse(charlie).
Goal: horse(charlie)
  offspring(charlie, Y) -> parent(Y, charlie) -> Y = bluebeard [TRUE]
  horse(bluebeard) [FACT -> TRUE]
=> TRUE

?- mammal(charlie).
mammal(charlie) <- horse(charlie) [TRUE] => TRUE
```

4.5 Output

```
?- horse(charlie).  
true.  
  
?- mammal(charlie).  
true.  
  
?- offspring(charlie, bluebeard).  
true.
```

Listing 4: Assignment 2 output

4.6 Discussion

Charlie is the offspring of Bluebeard (inverse of parent relation). Since Bluebeard is a horse, Charlie inherits the horse property via rule (2). Consequently, Charlie is also a mammal via rule (1). This demonstrates inverse relations and transitive rule chaining in Prolog.

5 Assignment 3 — Student Evaluation: Forward & Backward Chaining

5.1 Problem Summary

A Student Evaluation System defines three rules linking lecture attendance, studying, course coverage, good position, and first position. Given that a student attends all lectures, studies course books, and has no one with higher marks, the task is to prove the student will get first position using both forward and backward chaining.

5.2 FOPL Representation

Rule I: $\forall X \text{ (GoodPosition}(X) \wedge \text{NoHigherMarks}(X) \Rightarrow \text{FirstPosition}(X))$

Rule II: $\forall X \text{ (AttendsLectures}(X) \wedge \text{StudiesBooks}(X) \Rightarrow \text{CoversCourse}(X))$

Rule III: $\forall X \text{ (CoversCourse}(X) \Rightarrow \text{GoodPosition}(X))$

Facts: $\text{AttendsLectures}(\text{student})$, $\text{StudiesBooks}(\text{student})$, $\text{NoHigherMarks}(\text{student})$

5.3 Prolog Code (Backward Chaining)

```

1 attends_lectures(student).
2 studies_books(student).
3 no_higher_marks(student).
4
5 covers_course(X) :-
6     attends_lectures(X),
7     studies_books(X).
8
9 good_position(X) :-
10    covers_course(X).
11
12 first_position(X) :-
13    good_position(X),
14    no_higher_marks(X).
```

Listing 5: Student evaluation — backward chaining

5.4 Backward Chaining Trace

```

Goal: first_position(student)
|- good_position(student) AND no_higher_marks(student)
|   |
|   |                               |
|   |                               +-> FACT => TRUE
|   +-> covers_course(student)
|       +-> attends_lectures(student) [FACT]
|       +-> studies_books(student)    [FACT]
|       => TRUE
=> TRUE    (Q.E.D.)
```


5.5 Forward Chaining Trace

Step	Rule	Derivation
1	Rule II	$\begin{array}{l} \text{attends_lectures}(\text{student}) \\ \text{studies_books}(\text{student}) \end{array} \Rightarrow \text{covers_course}(\text{student}) \quad \wedge$
2	Rule III	$\text{covers_course}(\text{student}) \Rightarrow \text{good_position}(\text{student})$
3	Rule I	$\begin{array}{l} \text{good_position}(\text{student}) \\ \text{no_higher_marks}(\text{student}) \end{array} \Rightarrow \text{first_position}(\text{student}) \quad \blacksquare$

5.6 Forward Chaining in Prolog

```

1 :- dynamic derived/1.
2
3 forward_chain :-
4     ( attends_lectures(S), studies_books(S),
5       \+ derived(covers_course(S))
6     -> assert(derived(covers_course(S))),
7         format('Step 1 (Rule II): Derived covers_course(~w)~n',
8               [S])
9     ; true),
10    ( derived(covers_course(S)),
11      \+ derived(good_position(S))
12    -> assert(derived(good_position(S))),
13        format('Step 2 (Rule III): Derived good_position(~w)~n',
14              [S])
15    ; true),
16    ( derived(good_position(S)), no_higher_marks(S),
17      \+ derived(first_position(S))
18    -> assert(derived(first_position(S))),
19        format('Step 3 (Rule I): Derived first_position(~w)~n',
20              [S])
21    ; true).
```

Listing 6: Forward chaining using assert/1

5.7 Output

```

?- first_position(student).
true.

?- forward_chain.
Step 1 (Rule II): Derived covers_course(student)
Step 2 (Rule III): Derived good_position(student)
```

```
Step 3 (Rule I):   Derived first_position(student)
true.
```

Listing 7: Assignment 3 output

5.8 Discussion

Both inference strategies reach the same conclusion. Backward chaining decomposes the goal top-down into sub-goals resolved against facts. Forward chaining starts from facts and derives new conclusions bottom-up until the goal is reached. Both confirm: **the student will get first position.**

6 Summary

Table 1: Summary of assignments

Assignment	Key Concepts	Chain Depth	Strategy
1 (Courses)	Resolution, transitive rules	2 levels	Backward
2 (Horses)	Biconditional (inverse), inheritance	3 levels	Backward
3 (Student)	Forward + Backward comparison	3 levels	Both

7 Conclusion

This lab provided hands-on experience with First Order Predicate Logic and Prolog. Knowledge statements were systematically encoded in FOPL and translated into executable Prolog programs. Backward chaining was demonstrated as Prolog’s native inference strategy, while forward chaining was simulated using `assert/1`. Both strategies proved logically sound, arriving at identical conclusions for the Student Evaluation problem. The assignments highlighted Prolog’s strength in declarative knowledge representation and automated reasoning.

8 References

1. Bratko, I. (2012). *Prolog Programming for Artificial Intelligence* (4th ed.). Pearson.
2. Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
3. Clocksin, W. F., & Mellish, C. S. (2003). *Programming in Prolog* (5th ed.). Springer.
4. SWI-Prolog Documentation. <https://www.swi-prolog.org/pldoc/>